

A Prescriptive Pattern for Encoding Controlled Vocabulary Enumeration (CVE) for Ontological Data

E. Wray Johnson

LinkedIn: <https://www.linkedin.com/in/e-wray-j-6a56a1/>

Author's Note

This paper describes a novel formalization of controlled vocabulary enumeration (CVE)—a long-standing practice—into a repeatable encoding pattern for ontologies and linked data. The pattern is non-proprietary; encouraging its broad adoption is a primary purpose of this paper.

The pattern serves as one foundational primitive within **DataCentriX**—a comprehensive data-centric fabric whose full architecture extends well beyond what is documented here. While the prescriptive pattern is disclosed in detail hereafter, the broader design, capabilities, and scope of DataCentriX are intentionally omitted. Readers interested in exploring DataCentriX are encouraged to reach out to the author directly.

Abstract

This Controlled Vocabulary Enumeration (CVE) encoding pattern enforces closed value sets with governed semantics by integrating RDF, RDFS, OWL, SKOS, and DCMI Metadata Terms. This approach allows instance data to store only compact, authoritative codes while labels, definitions, history, and hierarchy evolve safely over time.

The pattern addresses pervasive enterprise and government challenges: schema drift, taxonomy proliferation, multilingual support, legacy data reconciliation, and free-text ambiguity. It does this by relying entirely on established standards, requiring no proprietary extensions or external constraint languages.

It enables:

- Deterministic validation through OWL datatype enumeration
- Stable code values decoupled from linguistic variation
- Governed semantics via SKOS labels and hierarchy
- Safe accommodation of vocabulary evolution over time
- Improved grounding and explainability for AI systems

The pattern is applicable across domains.

Examples throughout this paper illustrate the different real-world pressures motivating its design.

Introduction

Enumerations and code lists have long been fundamental to data systems—constraining values, enforcing correctness, and making intent explicit. As systems evolved from monolithic applications to distributed architectures,

these enumerations moved from hardcoded constants to shared controlled vocabularies, enabling reuse, governance, and consistent interpretation across organizational boundaries.

The semantic web and linked data communities adopted controlled vocabularies early, recognizing their value for data integration and interoperability. Yet implementing Controlled Vocabulary Enumeration (CVE) effectively in RDF/OWL has proven challenging. This paper documents a prescriptive pattern that addresses this challenge by binding controlled vocabulary codes to property ranges with enforceable constraints.

This pattern addresses the architectural gap between vocabulary description and value enforcement. It explains the problem (§1), describes the solution pattern overview (§2) and structural components (§3), explores semantic richness (§4), demonstrates capabilities through real-world examples (§5), establishes normative invariants (§6), provides standards alignment (§7), articulates storage and localization benefits (§8), explores value for AI systems (§9), validates through applied use (§10), and concludes with comparative analysis, applicability, and pattern summary (§11).

Note on terminology: The term "CVE" in this document refers to Controlled Vocabulary Enumeration and is explicitly disambiguated from the cybersecurity meaning (Common Vulnerabilities and Exposures).

1. Problem Statement

1.1 The Limits of Code Lists in Practice

Most systems rely on enumerations ("enums") to constrain values such as:

- Geolocation codes such as countries, states and airports
- Classification markings
- Workflow states
- Domain-specific codes

In practice, these enums are often:

- Hardcoded in application logic
- Duplicated across services
- Inconsistently validated
- Poorly documented
- Difficult to localize

This leads to schema drift, integration tax, and governance risk.

1.2 What SKOS Solves — and What It Does Not

SKOS provides an excellent model for controlled vocabularies:

- `skos:ConceptScheme` groups vocabularies
- `skos:Concept` defines individual terms
- `skos:notation` captures compact machine-oriented codes
- `skos:prefLabel` and `skos:altLabel` support multilingual presentation

However, SKOS is intentionally non-prescriptive by design—It prioritizes flexibility over constraint, leaving it to other standards (like OWL) to define validation rules.

This flexibility has a cost: SKOS does not provide a mechanism to bind the `skos:notation` values of a concept scheme to the allowed values of a specific property.

As a result, SKOS alone cannot:

- Enforce allowed values at ingest time
- Prevent typos or invalid literals
- Guarantee closed value sets

1.3 The OWL Enumeration Tradeoff

OWL supports value restriction via `owl:oneOf`, including enumeration of literal values.

While this enables strict validation, OWL enumerations:

- Lack governance metadata
- Do not support multilingual labels
- Provide no built-in provenance

Used alone, OWL enumerations become isolated validation mechanisms—opaque to governance inquiries, divorced from vocabulary maintenance, and brittle when evolution requires coordinated updates.

What is needed is a pattern that combines both approaches: SKOS's governance and semantic richness with OWL's enforcement capabilities.

2. Solution Pattern Overview

The solution is a pattern that pairs SKOS's semantic governance capabilities with OWL's enforcement mechanisms. Rather than choosing between them, this pattern composes them into a coherent whole:

- A controlled **vocabulary** (SKOS)—maintains semantic richness, labels, and governance metadata
- A custom **enumeration datatype** (OWL)—constrains which values are allowed
- An **explicit linkage** between them (DCMI Metadata Terms)—grounds the datatype in the vocabulary source
- **Property ranges** bound to the datatype (RDFS)—enforces the constraint at the data layer

This composition addresses the architectural gap identified in §1. SKOS provides governance without enforcement; OWL provides enforcement without governance. The pattern eliminates this tradeoff by composing both.

Key consequence: Instance data stores only compact literal values (the codes), while all semantic meaning—labels, definitions, hierarchy, and governance—is maintained centrally in the vocabulary. This minimizes storage, simplifies validation, and enables safe vocabulary evolution without disrupting instance data.

Sections §3-4 detail the structural components and semantic capabilities of this composition.

Architectural Significance

This CVE pattern supplies a missing architectural primitive:

The ability to bind controlled vocabulary codes (`skos:notation`) to property ranges with enforceable constraints.

This capability does not exist in SKOS or OWL alone. By composing SKOS, OWL, and DCMI Metadata Terms, the pattern fills this gap and enables practical governance-with-enforcement in RDF/OWL systems.

3. Solution Pattern Structural Components

3.1 SKOS ConceptScheme

The concept scheme is the authoritative vocabulary:

- Owns provenance and governance
- Defines lifecycle and stewardship
- Contains all semantic context

This provides the semantic foundation, but does not enforce which values may appear in instance data.

3.2 SKOS Concepts

Each allowed value corresponds to exactly one concept:

- `rdf:type skos:Concept`
- `skos:inScheme` → the concept scheme
- `skos:notation` → the literal value
- Optional labels, definitions, and notes

SKOS explicitly supports **typed literals** for `skos:notation` to distinguish different kinds of codes (e.g., country codes, product codes, classification identifiers). However, SKOS is intentionally non-prescriptive (see §1.2) in how such datatypes are enforced in instance data.

This encoding pattern builds on this intended flexibility by pairing typed `skos:notation` values with OWL datatype enumeration and explicit `rdfs:range` constraints. This supplies closed value enforcement while remaining fully compliant with SKOS, RDF, and OWL, and without introducing an additional constraint language. No external constraint language (e.g., SHACL) is required.

3.3 Custom CVE Datatype

The CVE datatype is the binding and enforcement point that addresses SKOS's lack of value constraint mechanisms. This CVE encoding pattern aligns with W3C best practices; the Data Cube specification explicitly recommends creating a custom class and datatype for code lists and binding them via `rdfs:range` to enable validation using standard RDF tools.

It has the following characteristics:

- `rdf:type rdfs:Datatype`
- `owl:equivalentClass` → `owl:oneOf` (... literals ...)
- `dc:source` → authoritative `skos:ConceptScheme`

Beyond `dc:source`, no additional linkage properties are required for correctness, validation, or governance.

Note: `rdfs:seeAlso` was evaluated but not adopted because it is non-normative, provides no enforceable semantics, and introduces ambiguity. In contrast, `dc:source` cleanly asserts a single authoritative source without implying inference.

3.4 Datatype Property

Each governed property is defined as an `owl:DatatypeProperty` :

- Instance values are literals (strings, numbers, etc.)
- `rdfs:range` points to a custom CVE datatype

This ensures enforcement occurs at the data layer.

4. Semantic Richness Using OWL and SKOS

Beyond the core CVE enforcement pattern, semantic richness can be layered into controlled vocabularies through multiple mechanisms. This section explores how the pattern supports deeper semantic modeling: explicit relationships and axioms via OWL, vocabulary organization and hierarchy via SKOS, and real-world semantic pressures illustrated through country code examples.

Note on Provenance Annotation: Provenance metadata—such as creation date, responsible party, change history, or derivation—can be attached to the concept scheme (§3.1), individual concepts (§3.2), the custom datatype (§3.3), or governed properties (§3.4) using standard OWL annotation properties (e.g., `dc:date`, `dc:creator`, `prov:wasDerivedFrom`, `rdfs:comment`). Such annotation is standard practice in ontology engineering and is not detailed in this document. The CVE pattern itself is agnostic to provenance mechanism; vocabulary stewards may employ whichever annotation strategy fits their governance requirements.

4.1 Synonyms and Antonyms (OWL)

True synonymy and opposition are modeled using OWL constructs such as `owl:equivalentClass` and `owl:disjointWith`.

4.2 Vocabulary Organization and Hierarchy (SKOS)

SKOS supports organized knowledge representation through navigation and hierarchical relationships via `skos:broader` and `skos:narrower`, allowing vocabularies to reflect both flat code lists and structured hierarchies without affecting enforcement.

5. Real-World Usage Examples

The mechanisms introduced in §4 address real-world semantic pressures. This section demonstrates how OWL and SKOS constructs combine to handle vocabulary evolution, historical continuity, and organizational complexity through concrete country code examples.

5.1 Turkey / Türkiye (TR) — Renaming, Language, and Diacritics

Turkey illustrates lexical evolution without identifier change. The ISO 3166-1 alpha-2 code `TR` remains stable while the country's official name and preferred spelling evolved.

A concept with notation `"TR"` may include:

- `skos:notation "TR"^^<http://id.iso.org/iso3166/alpha2>`
- `skos:prefLabel "Türkiye"@tr`

- `skos:altLabel "Turkey"@en`
- `skos:hiddenLabel "Türkiye"@en`
- `skos:hiddenLabel "Türkiye"@en`
- `skos:hiddenLabel "TUR"@en`

Context: The Republic of Turkey officially changed its English-language country name to the Republic of Türkiye in May 2022 following a request to the United Nations. This rebranding was intended to better represent Turkish culture and identity and to avoid unintended associations in English usage.

This case justifies:

- Separation of identifiers from labels
- Tolerance of diacritic loss and OCR noise
- Stable storage despite political or linguistic change

5.2 Democratic Republic of the Congo (CD) — Historical Renaming

The Democratic Republic of the Congo demonstrates long-term historical renaming while preserving identifier continuity.

- Legal CVE value: `CD`
- Historical name captured explicitly as: `skos:hiddenLabel "Zaire"@en`

Legacy documents and datasets frequently reference Zaire. Treating this name as a hidden label allows reconciliation and search without reintroducing an obsolete identifier into stored data.

This case justifies:

- Use of `skos:hiddenLabel` for historically valid but no longer authoritative names
- Preservation of historical meaning without semantic regression

5.3 South Sudan (SS) — Emergence of a New Country

South Sudan illustrates state emergence, not renaming. In 2011, South Sudan became an independent nation with its own ISO code `SS`, distinct from Sudan (`SD`).

This is not a synonym or alias case. **SS must not be modeled as a hidden label of SD.**

This case justifies:

- Introduction of new CVE values without collapsing historical data
- Clear semantic fission rather than normalization
- Support for geopolitical change over time

5.4 Geographic Hierarchy: Continents and Country Codes

Continents and country codes illustrate broader/narrower semantics rather than identifier substitution.

- Continents use a small, separate CVE domain (e.g., `AF`, `EU`, `AS`)
- Country codes use ISO 3166-1 alpha-2 codes

SKOS `skos:broader` / `skos:narrower` relationships may be used to organize country codes under continents for navigation and grouping (e.g., Africa → Democratic Republic of the Congo), while CVE encoding pattern enforcement

ensures that continent codes and country codes remain distinct and unambiguous.

This case justifies:

- Hierarchical semantics without polluting value spaces
- Safe aggregation and regional reasoning
- Clean separation between geography and identifiers

5.5 ISO 3166-1 Multi-Scheme Linking: Alpha-2 and Alpha-3 Codes

Organizations often maintain parallel code lists for the same domain using different standards. ISO 3166-1 provides both alpha-2 codes (e.g., `TR`, `CD`, `SS`) and alpha-3 codes (e.g., `TUR`, `COD`, `SSD`) for the same countries.

This CVE encoding pattern enables two separate concept schemes—one for alpha-2, one for alpha-3—while linking concepts across schemes using `owl:sameAs`:

ISO 3166-1 Alpha-2 Concept Scheme:

```
iso3166alpha2:TR
  rdf:type skos:Concept
  skos:notation "TR"^^iso3166:alpha2
  skos:prefLabel "Türkiye"@tr
  owl:sameAs iso3166alpha3:TUR
```

ISO 3166-1 Alpha-3 Concept Scheme:

```
iso3166alpha3:TUR
  rdf:type skos:Concept
  skos:notation "TUR"^^iso3166:alpha3
  skos:prefLabel "Türkiye"@tr
  owl:sameAs iso3166alpha2:TR
```

Each scheme enforces its own CVE datatype and value set independently. The `owl:sameAs` linkage enables:

- **Cross-scheme reconciliation** — Systems using alpha-2 can discover equivalent alpha-3 representations
- **Schema flexibility** — Different data stores may use different code standards without breaking interoperability
- **Safe expansion** — New code standards (e.g., numeric codes) can be added as additional schemes, linked to existing ones
- **Governance clarity** — Each scheme maintains independent versioning and stewardship while preserving semantic equivalence

This case justifies:

- Parallel standards as separate but linked CVE domains
- `owl:sameAs` for explicit semantic equivalence across vocabularies
- Schema-driven flexibility without sacrificing validation per scheme

Together, these examples show that CVE supports renaming, historical continuity, state emergence, and hierarchical organization simultaneously—without weakening validation or introducing ambiguity. This allows

systems to be forgiving at the edges (ingestion, search, and interpretation) while remaining strict at the core (storage and validation).

6. Normative Invariants

The semantic capabilities just described are grounded in explicit, enforceable rules. This CVE encoding pattern enforces the following invariants:

1. **Closed Value Set** — All allowed values must appear in `owl:oneOf`
 2. **1-to-1 Mapping** — Each `owl:oneOf` literal must correspond to exactly one `skos:notation` value
 3. **Authoritative Source** — The `dc:source` concept scheme is the sole source of meaning
 4. **Literal-Centric Instance Data** — Instance data never stores concept IRIs
 5. **No Implicit Inference** — All linkages are explicit and deterministic
-

7. Standards Alignment

Key Standards Endorsing This Approach

- **W3C SKOS Reference** explicitly positions SKOS as minimal, intended for composition with OWL and complementary standards.
- **W3C SKOS and OWL** discusses patterns where OWL provides formal constraints while SKOS provides organization and labels.
- **RDF 1.1** defines datatypes as first-class constructs for custom definition and reuse.
- **OWL 2** provides `owl:oneOf`, datatype restrictions, and disjointness—designed to complement SKOS.
- **SKOS documentation** confirms SKOS's architectural gap: custom notations "do not provide a mechanism to describe the internal structure of such classification codes."
- **W3C Data Cube** explicitly recommends creating custom datatypes for code lists, binding them via `rdfs:range`, and enabling validation without custom tooling.

This CVE pattern is a formalization of existing W3C best practice, not an extension.

8. Storage and Localization Benefits

The practical advantages of this pattern become evident in production systems managing large-scale data governance. These benefits are particularly significant in complex architectures like **DataCentriX**, where efficiency and centralized governance directly impact system scalability and operational cost.

8.1 Compact Storage

Instance data stores only short, standardized codes (e.g., airport codes).

This:

- Minimizes storage
- Reduces index size

- Improves cache locality
- Lowers IO cost

8.2 Centralized Localization

Human-readable names are stored once in the vocabulary:

- Language-tagged labels
- Jurisdiction-specific naming
- Multiple display conventions

No instance data duplication is required.

8.3 Governance, Security, and Auditability

CVE vocabularies are first-class governed data:

- Versioned
- Access-controlled
- Auditable

This enables:

- Need-to-know filtering of allowed values
- Phased rollout or deprecation
- Compliance and accreditation narratives

9. Value Propositions: AI and Machine Understanding

This CVE pattern improves AI behavior by grounding machine interpretation in explicit, bounded semantics. It separates stable identifiers from linguistic surface forms, enabling normalization of noisy input to authoritative codes before reasoning occurs. AI systems gain:

- **Clear domain boundaries** that reduce hallucination
- **Stable identifiers** suitable for training and reasoning
- **Explicit relationships** that support safer inference
- **Human-aligned outputs** from governed SKOS labels

In effect, CVE makes the meaning AI operates on less ambiguous and more durable.

9.1 Boundaries, Detection, and Signal Clarity

Because values are constrained to a closed, enumerated set:

- AI systems can reliably distinguish what a value **can be** from what it **cannot be**
- Classification and extraction models operate against known domains
- Hallucinated or out-of-vocabulary values can be detected and rejected

The encoding pattern cleanly separates **signal** (the stored code/notation) from **surface form** (labels, synonyms, legacy names, misspellings). This means:

- Multiple lexical forms map to a single canonical meaning
- Training and inference can normalize noisy inputs
- Reasoning operates over stable identifiers, not text strings

This is especially important for large language models, which otherwise tend to generalize beyond intended domains.

9.2 Information Extraction and Conceptual Clarity

When applied to unstructured or semi-structured data, this CVE encoding pattern enables grounding extracted terms to authoritative vocabularies and normalization across documents, languages, and time. Hidden labels (`skos:hiddenLabel`) are particularly valuable for AI-assisted ingestion.

OWL-level semantics (equivalence, disjointness, hierarchy) provide AI systems with explicit conceptual boundaries and known incompatibilities. This reduces spurious correlations and improves explainability when AI systems classify, recommend, or summarize information.

9.3 Human–AI Alignment

Because SKOS labels and notes remain human-readable while codes remain machine-stable, AI outputs can be rendered in user-appropriate language, explanations can reference authoritative definitions, and human review and correction are simplified. The result is tighter alignment between human understanding and machine reasoning.

10. Applied Use: Security Marking Ontology and DataCentriX

This CVE encoding pattern has been validated through real-world application. The security marking ontology described here demonstrates the pattern's effectiveness, and more broadly, these mechanisms are employed within **DataCentriX**, a production data governance architecture that manages complex, multi-tenant systems with varying access and classification requirements.

Security Marking Specification

This CVE encoding pattern has been applied in a proposed security marking ontology developed in support of the DIOWG/NSOWG. The ontology is a direct, 1:1 RDF/OWL encoding of the DNI's security marking specification, which is itself defined authoritatively in XML/XSD. The RDF/OWL representation is produced via programmatic transformation of that specification, rather than manual reinterpretation or semantic embellishment.

The resulting ontology is published in multiple RDF serializations—including **Turtle**, **JSON-LD**, and **RDF/XML**—ensuring broad interoperability across tools, platforms, and processing environments while preserving a single, consistent semantic model.

This approach preserves strict fidelity to the authoritative marking standard while making the structure machine-actionable. This CVE encoding pattern is used to represent marking codes as closed, enforceable value sets, ensuring that only valid, specification-defined markings may appear in data.

Encoding Approach

Marking codes are modeled as CVE-enforced OWL datatypes. Human-readable markings, abbreviations, and historical variants are represented using SKOS labels (including `skos:hiddenLabel`). Semantic relationships are expressed explicitly in OWL. The ontology exhibits clear traceability, availability in multiple RDF encodings, and suitability for both automated validation and human interpretation.

AI and Automation Implications

By encoding security markings as closed, governed value sets with explicit semantics, this CVE encoding pattern provides AI systems with a **grounded semantic substrate**. Markings extracted from documents, metadata, or text can be normalized to authoritative codes before reasoning occurs, reducing ambiguity and preventing hallucination.

This allows downstream automation—classification, filtering, summarization, and analysis—to operate over **explicit meaning** rather than inferred text, which is essential in high-assurance environments.

Crucially, this grounding enables AI to operate in accordance with a user's Need-to-Know (NTK). Because security markings are represented as enforceable, machine-interpretable codes rather than free text, AI systems can deterministically filter, redact, summarize, or reason over information based on the markings a user is authorized to see. This allows **AI behavior itself—not just data access—to be aligned with policy**, enabling explainable, policy-constrained automation in security-sensitive environments.

11. Comparison, Applicability, and Pattern Summary

Comparative Positioning

Approach	Enforcement	Governance	Localization	Drift Risk
Hardcoded enums	✗	✗	✗	High
SKOS only	✗	✓	✓	Medium
OWL enum only	✓	✗	✗	Medium
This CVE pattern	✓	✓	✓	Low

Suitable Application Domains

This CVE pattern is suitable for:

- Enterprise data platforms
- Government and regulated environments
- Multilingual systems
- Schema-driven UIs
- Hybrid structured / unstructured data architectures

It is a foundational pattern for data-centric system design.

Pattern Characteristics and Value

This **CVE pattern** is a pragmatic, production-grade solution for governed enumerations. It has been proven in real-world deployments like **DataCentriX**, where it enables complex governance at scale. The pattern:

- Enforces correctness at the data layer
- Minimizes storage through compact literal codes
- Enables centralized localization across systems
- Supports schema-driven automation and validation
- Aligns with regulated, high-assurance data-centric architectures

By composing existing W3C standards instead of extending them, this CVE pattern demonstrates how governed enumeration can be operationalized in real systems without proprietary ownership or external constraint languages. It is a foundational design principle for systems requiring both strict correctness and flexible governance.



DataCentriX

A data-centric fabric that treats meaning, validation, and governance as intrinsic properties of the data itself.

Author: E. Wray Johnson

LinkedIn: <https://www.linkedin.com/in/e-wray-j-6a56a1/>